

Name: Nilanjan Ghosh

College: SRM Institute of Science and Technology

Course: Java FSE Digital Nurture

Week 1

Exercise 1: Mocking and Stubbing Using Mockito

Aim

To understand and implement mocking and stubbing using Mockito for testing services that depend on external APIs.

Objective

The objective of this exercise is to isolate the service layer from external dependencies by creating mock objects and stubbing predefined responses during unit testing.

Software Requirements

- Java Development Kit (JDK)
 - Apache Maven
 - Visual Studio Code
 - JUnit
 - Mockito
-

Theory

Mockito is a Java framework used for creating mock objects during unit testing. Mock objects simulate the behavior of real objects and help isolate the component being tested.

Mocking

Mocking refers to creating a fake implementation of a dependency.

Example:

```
ExternalApi mockApi = mock(ExternalApi.class);
```

Stubbing

Stubbing defines the behavior of a mocked method.

Example:

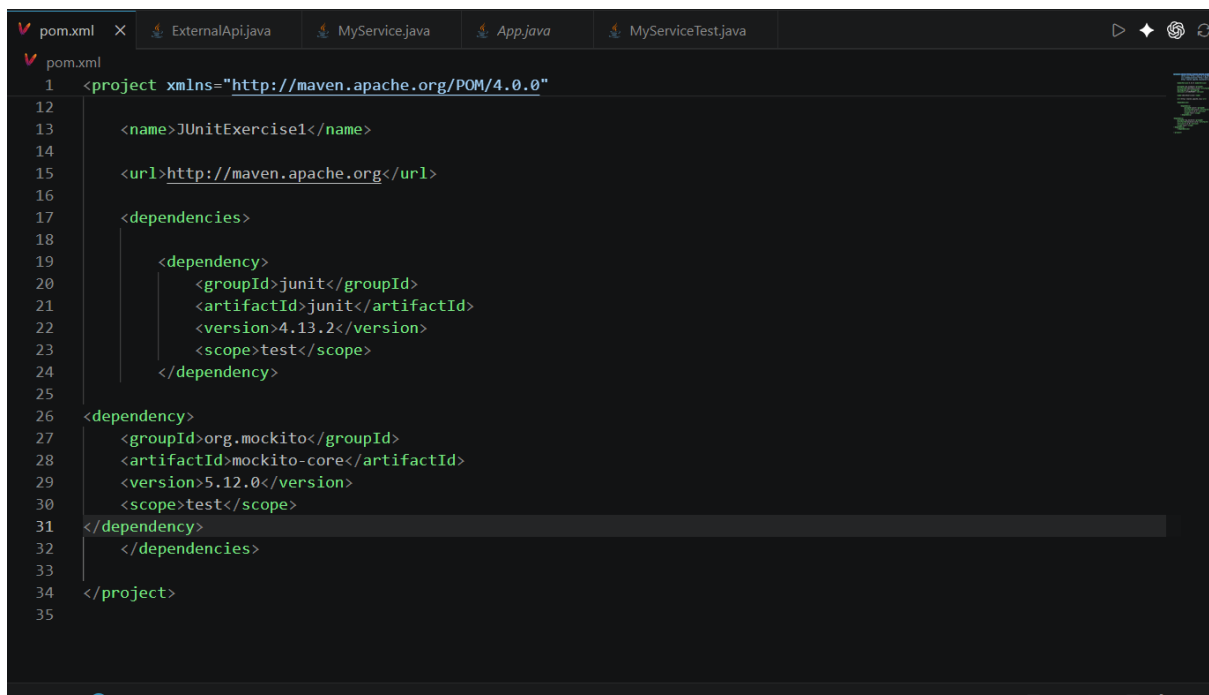
```
when(mockApi.getData()).thenReturn("Mock Data");
```

Procedure

Step 1: Add Mockito Dependency

The Mockito dependency was added to the Maven project through the pom.xml file.

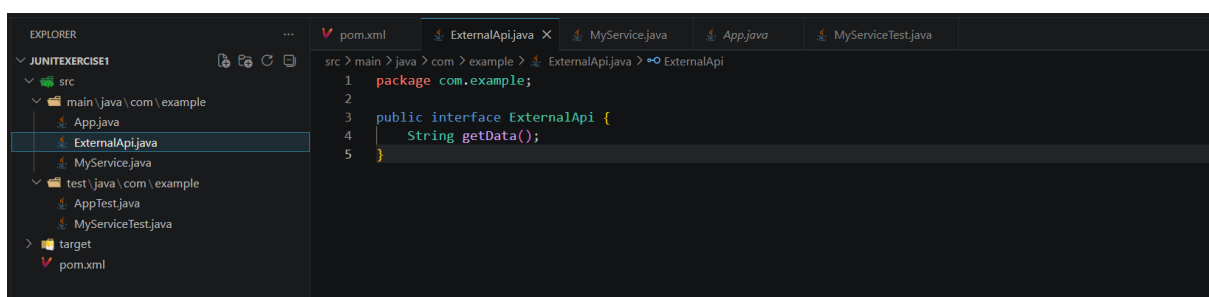
Screenshot 1:



Step 2: Create External API Interface

An interface named ExternalApi was created to represent an external service dependency.

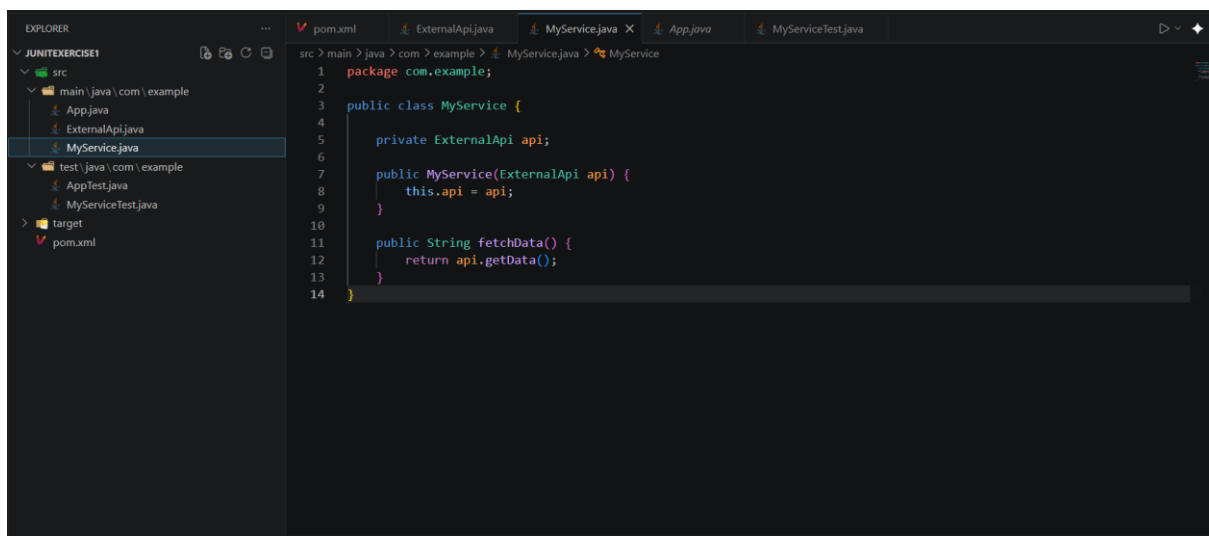
Screenshot 2:



Step 3: Create Service Class

A service class named `MyService` was created. The service depends on the `ExternalApi` interface and retrieves data through it.

Screenshot 3:



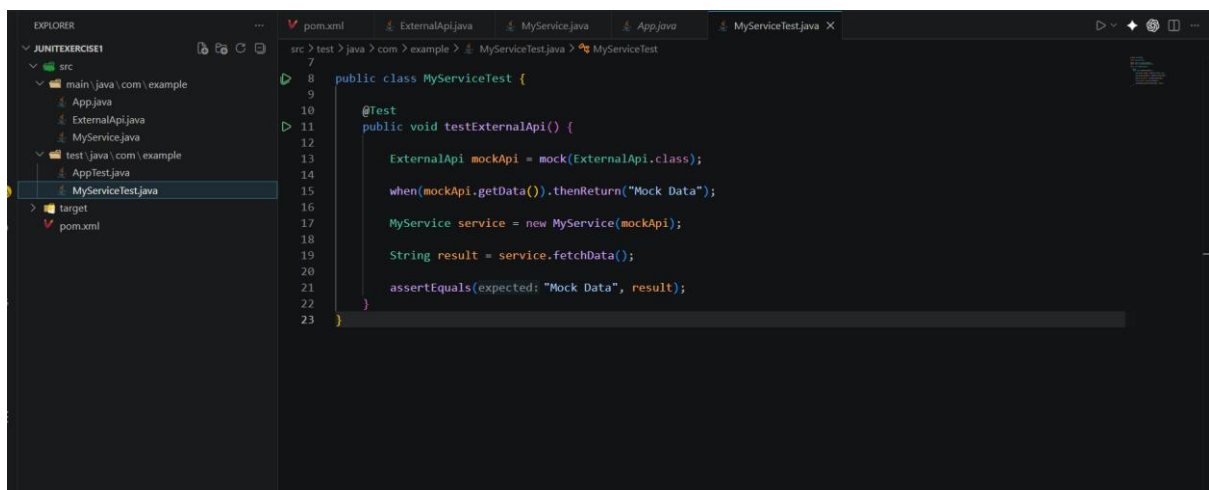
Step 4: Create Test Class

A JUnit test class named `MyServiceTest` was created.

The test performs the following:

- Creates a mock object.
- Stubs the `getData()` method.
- Injects the mock object into `MyService`.
- Verifies the returned result using assertions.

Screenshot 4:

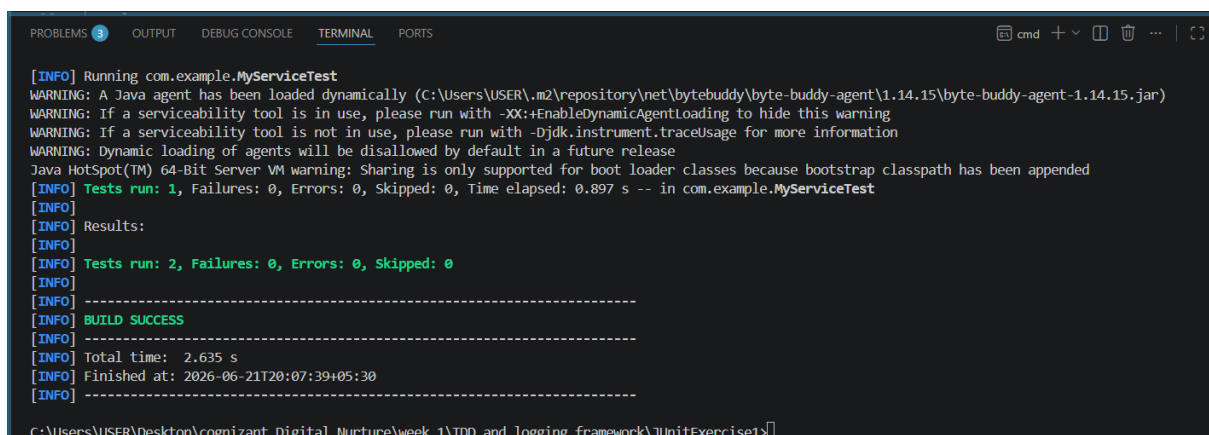


Step 5: Execute Test Cases

The Maven test command was executed.

mvn test

Screenshot 5:



Output

The mocked API returned the predefined value "Mock Data" and the assertion passed successfully.

Output:

Tests run: 1

Failures: 0

Errors: 0

Screenshot 6:

Result

Conclusion

Exercise 2: Verifying Interactions Using Mockito

Aim

To verify that a method is invoked with the expected interaction using Mockito.

Objective

The objective of this exercise is to use Mockito's verification feature to ensure that a dependent object's method is called during execution.

Software Requirements

- Java Development Kit (JDK)
 - Apache Maven
 - Visual Studio Code
 - JUnit
 - Mockito
-

Theory

Mockito provides the `verify()` method to confirm that a particular method was called on a mock object.

Verification helps ensure that the application interacts correctly with its dependencies.

Example:

```
verify(mockApi).getData();
```

This statement confirms that the `getData()` method was invoked on the mocked API object.

Procedure

Step 1: Create a Mock Object

A mock implementation of the `ExternalApi` interface was created using Mockito.

Screenshot 1:

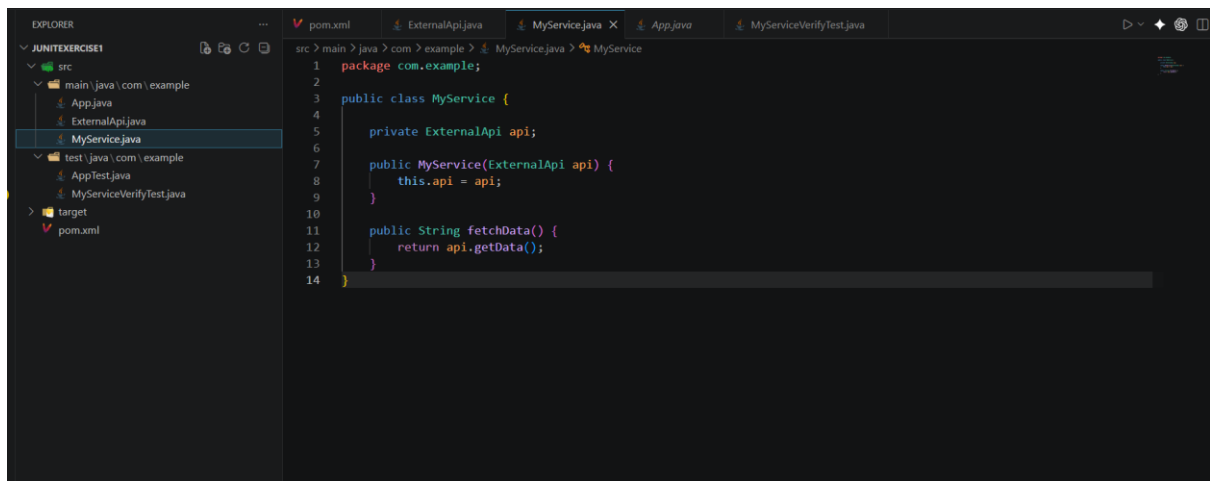


```
src > test > java > com > example > MyServiceVerifyTest.java > MyServiceVerifyTest
1  package com.example;
2
3  import org.junit.Test;
4
5  import static org.mockito.Mockito.*;
6
7  public class MyServiceVerifyTest {
8
9      @Test
10     public void testVerifyInteraction() {
11
12         ExternalApi mockApi = mock(classToMock: ExternalApi.class);
13
14         MyService service = new MyService(mockApi);
15
16         service.fetchData();
17
18         verify(mockApi).getData();
19     }
20 }
```

Step 2: Create Service Object

An instance of MyService was created by injecting the mocked API object.

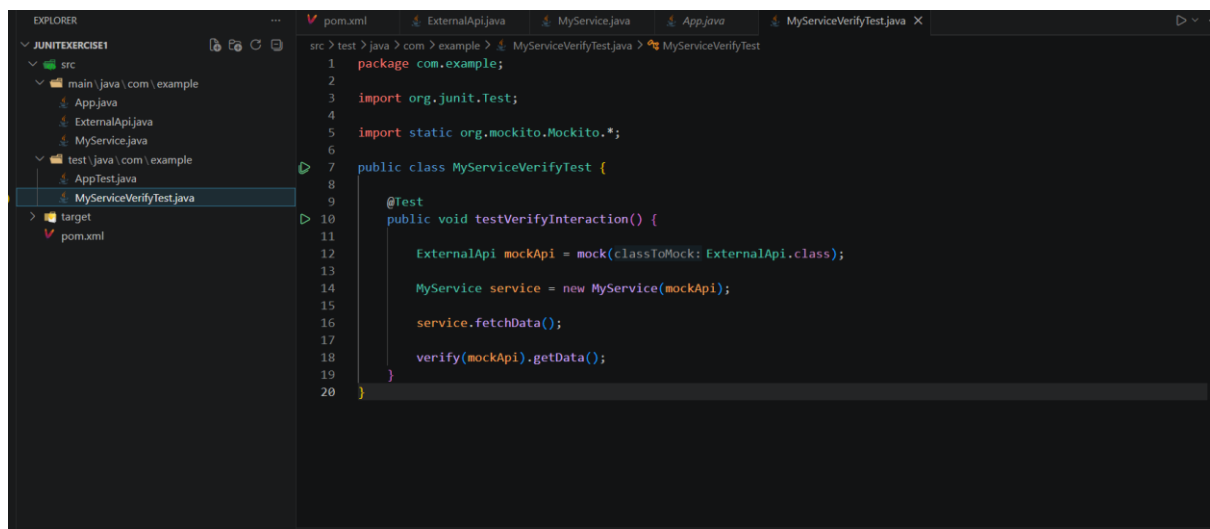
Screenshot 2:



Step 3: Invoke Method

The fetchData() method was called from the service class.

Screenshot 3:



Step 4: Verify Interaction

Mockito's `verify()` method was used to confirm that `getData()` was called.

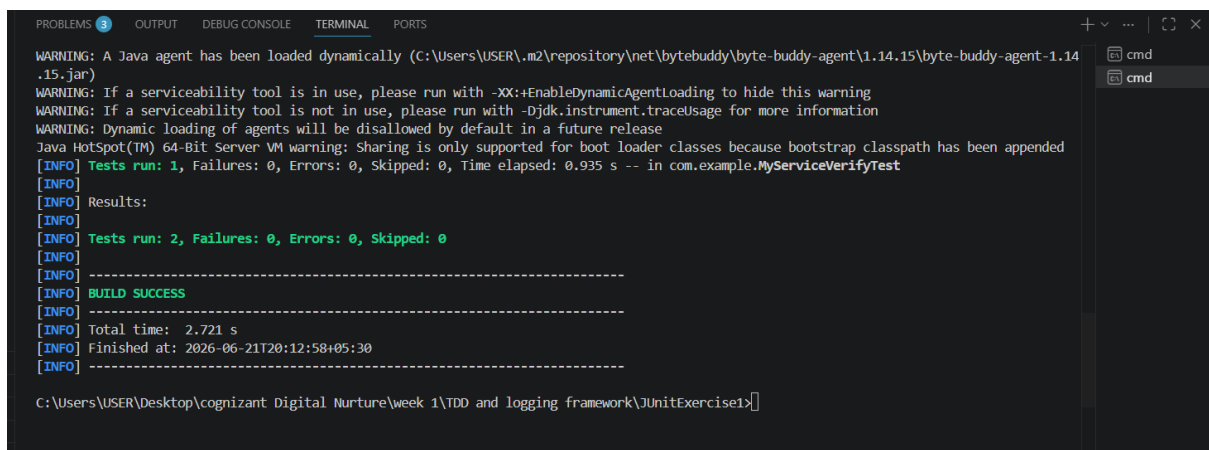
```
verify(mockApi).getData();
```

Step 5: Execute Test

The Maven test command was executed.

```
mvn test
```

Screenshot 5:

A screenshot of an IDE terminal window. The terminal shows the output of a Maven test command. It starts with several warnings about a Java agent and dynamic loading. Then, it shows the test results: 'Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.935 s -- in com.example.MyServiceVerifyTest'. This is followed by 'Results:' and 'Tests run: 2, Failures: 0, Errors: 0, Skipped: 0'. The terminal concludes with 'BUILD SUCCESS', 'Total time: 2.721 s', and 'Finished at: 2026-06-21T20:12:58+05:30'. The command prompt at the bottom shows the current directory as 'C:\Users\USER\Desktop\cognizant Digital Nurture\week 1\TDD and logging framework\JUnitExercise1>'.

```
PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS
WARNING: A Java agent has been loaded dynamically (C:\Users\USER\.m2\repository\net\bytebuddy\byte-buddy-agent\1.14.15\byte-buddy-agent-1.14.15.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.935 s -- in com.example.MyServiceVerifyTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.721 s
[INFO] Finished at: 2026-06-21T20:12:58+05:30
[INFO] -----
C:\Users\USER\Desktop\cognizant Digital Nurture\week 1\TDD and logging framework\JUnitExercise1>
```

Output

The verification was successful and all test cases passed.

Tests run: 1

Failures: 0

Errors: 0

BUILD SUCCESS

Result

Mockito successfully verified that the `getData()` method was invoked on the mocked `ExternalApi` object.

Conclusion

This exercise demonstrated the use of Mockito's verification mechanism to confirm interactions between application components. Verifying method calls ensures that dependencies are being used correctly and improves the reliability of unit tests.